

How Accurately Can GPT-4 Generate Automated Feedback on Code Readability?

Anon Author 1
Anonymous Institution
City, Anonymous Country
anon@anon.com

Anon Author 2
Anonymous Institution
City, Anonymous Country
anon@anon.com

Anon Author 3
Anonymous Institution
City, Anonymous Country
anon@anon.com

Anon Author 4
Anonymous Institution
City, Anonymous Country
anon@anon.com

Abstract

Providing timely and consistent formative feedback on code readability is challenging in programming education, particularly in courses with hundreds of students. In this study, we explored the potential of GPT-4 to generate automated feedback on code readability for students in a CS1 Java assignment. We developed rubrics based on three key code readability criteria—naming, comments, and formatting. We further defined six criteria of feedback quality and incorporated these into GPT-4 prompts to guide feedback generation. To evaluate the effectiveness of the feedback, three evaluators analyzed five feedback examples generated by GPT-4, assessing them against our predefined rubrics. Our results were mixed: while some feedback closely aligned with the rubrics, offering valuable insights, other examples fell short in providing constructive guidance. This study highlights the potential and limitations of using Large Language Models like GPT-4 to generate scalable and formative feedback on code readability. Future research could refine these methods to improve feedback consistency and quality.

ACM Reference Format:

Anon Author 1, Anon Author 2, Anon Author 3, and Anon Author 4. 2018. How Accurately Can GPT-4 Generate Automated Feedback on Code Readability?. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 2 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Feedback is defined as information that highlights the gap between a learner's current knowledge level and the desired target [6]. Feedback is one of the most critical interventions for improving student learning [4]. For example, students receiving immediate feedback have more significant learning gains than students who do not [10]. Given the considerable disparity in the teacher-to-student ratio in typical classrooms, students claim that the feedback they receive

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
<https://doi.org/XXXXXXX.XXXXXXX>

often lacks in terms of quantity as well as quality [5]. Moreover, it is impractical for instructors to provide individualized feedback to each student, especially in massive online learning environments [10]. As Large Language Models (LLMs) such as GPT-4 are becoming prevalent, they have been used to provide automated feedback, thereby solving the challenges instructors face in providing manual feedback [3]. However, previous studies have not investigated the quality of automated feedback focused on student code readability. As a critical factor in code quality, code readability defines how easy a program is for humans to understand [2]. Therefore, this study aims to explore the performance of GPT-4 in generating automated feedback on student code readability and is guided by the following research question: **How accurate is the automated feedback generated by GPT-4 on student code readability?**

2 Methodology

2.1 Dataset

This study used the Menagerie dataset [1], which includes a second-semester CS1 assignment that ran over four academic years (18/19 - 21/22). The dataset consists of 667 total submissions. It collects students' open-ended programming solutions on an assignment to develop a predator/prey simulator. In this study, we work with 40 students' submissions of the Plant class.

2.2 Evaluation of Generated Feedback

We defined two sets of rubrics to evaluate the quality of the generated feedback. First, we constructed a code readability rubric by analyzing student code and reviewing relevant literature, and focused on the following three aspects. We chose them as they are typically essential aspects of code readability that generally have agreed-upon definitions that can be adapted for our prompts.

- Naming - including meaningful function and variable names, and the correct use of Java naming conventions.
- Comments - including docstrings related to the code they describe, do not state the obvious and contain no spelling or grammar errors [8].
- Formatting - correctly using whitespace between operators, blank lines between functions, and consistent indentation throughout their code [9].

Second, to evaluate the quality of feedback, we adapted Shute's [7] good feedback guidelines; some aspects, such as "not providing a

grade in feedback” were out of scope for our study. These guidelines were included in the prompt given to GPT-4:

- The feedback should be focused on the task, not the learner.
- The feedback should use “praise” sparingly, if at all.
- Do not give normative comparisons in the feedback.
- Do not present feedback that discourages the learner.
- Each point of feedback should link to the student code.
- Feedback should be consistent across students’ code.
- Feedback should be accurate.

2.3 Feedback Generation

We used GPT-4 to generate feedback for randomly selected 40 samples of student code. The provided prompts include two sections. First, the guidelines for good feedback were provided. Second, the definitions of each aspect of code readability were provided, with the corresponding examples. For our work-in-progress evaluation, we sampled five feedback samples and evaluated them using the criteria defined in Section 2.2.

3 Results

Of the five generated pieces of feedback we evaluated, most met our feedback criteria, with only one piece of feedback not providing an example of the error found in the student’s code and stated “...Ensure the whole code maintains a consistent level of indentation. Looking at your if and else code blocks, the code could be more readable with consistent indentation...”.

GPT-4 generated feedback on the meaningfulness of variable names in all five instances, but in one case, we deemed this incorrect. GPT-4 struggled the most when providing feedback on meaningful function names, giving incorrect feedback in most cases. In one instance, GPT-4 highlighted a Student class. It offered the feedback “... in your ‘Student’ class, if you had a boolean to track enrollment status, a clearer name would be ‘isStudentEnrolled’. ...” an example Student class was provided as part of the prompt. However, all classes submitted to GPT-4 only contained a Plant class and had no reference to Student classes.

For our comments criteria, GPT-4 failed to spot a minor spelling mistake in the documentation. It provided correct and complete feedback for the rest, including detecting and giving feedback on a copy-and-paste error, providing the input: “The comments in the giveBirth() method refers to the instance of this class as ‘Sheeps’. It appears to be a copy-paste error, ...”. GPT-4 struggled the most with our formatting criteria, with most feedback being incorrect or incomplete. The correct use of whitespace is typically subjective, with different people, organizations, and programming languages having different standards, which could explain why GPT-4 struggled to provide feedback on the student’s format.

While some of our results are promising, this is an initial evaluation of a small sample of GPT-4-generated pieces of feedback, and further research and evaluation are required to fully identify how well LLMs can provide feedback on students’ code readability.

4 Conclusion

We explored the potential of GPT-4 to generate formative feedback on code readability, focusing on three key criteria: naming,

comments, and formatting. The results revealed that while GPT-4 demonstrates promise in providing meaningful and actionable feedback, there are still limitations that need to be addressed.

One of the primary findings was the variability in the quality of feedback generated by GPT-4. While some generated feedback adhered closely to established rubrics and provided useful suggestions, such as the recommendation of renaming variables for clarity, some examples were less effective. Moreover, some generated feedback was inconsistent, offering contradictory advice on method naming without maintaining a clear rationale. These shortcomings indicate the need for more refined feedback guidelines, code readability definition, and prompt engineering.

Despite these limitations, the findings suggested that GPT-4 can support the learning process by offering timely, scalable feedback. Its capacity to generate feedback across large class sizes and provide students with insights into their code can alleviate the burden on instructors and offer more consistent feedback than human feedback. However, the variability in the feedback quality underscores the importance of further investigation into how the prompts can be optimized for this task.

In the future, we plan to develop a more comprehensive and detailed operational definition for code readability, incorporating additional criteria such as assessing code logic and documentation quality. A more thorough definition of these aspects will guide GPT-4 to generate more precise feedback. In this study, we manually evaluated five pieces of feedback generated by GPT-4. Moving forward, we aim to conduct a broader evaluation across more examples to gain deeper insights into the strengths and limitations of GPT-4’s automated feedback. This will involve assessing its performance across various programming assignments and readability issues.

References

- [1] Marcus Messer and Neil C. C. Brown and Michael Kölling and Miaojing Shi. 2024. How Consistent Are Humans When Grading Programming Assignments?
- [2] Raymond P.L. Buse and Westley R. Weimer. 2010. Learning a Metric for Code Readability. *IEEE Transactions on Software Engineering* 36, 4 (July 2010), 546–558. <https://doi.org/10.1109/TSE.2009.70>
- [3] Wei Dai, Jionghao Lin, Hua Jin, Tongguang Li, Yi-Shan Tsai, Dragan Gašević, and Guanliang Chen. 2023. Can Large Language Models Provide Feedback to Students? A Case Study on ChatGPT. In *2023 IEEE International Conf. on Advanced Learning Technologies (ICALT)*. 323–325. <https://doi.org/10.1109/ICALT58122.2023.00100>
- [4] John Hattie and Helen Timperley. 2007. The power of feedback. *Review of educational research* 77, 1 (2007), 81–112.
- [5] KL Krause, Robyn Hartley, Richard James, and Craig McInnis. 2005. The first year experience in Australian universities: Findings from a decade of national studies.
- [6] Elizabeth K. Molloy and David Boud. 2014. *Feedback Models for Learning, Teaching and Performance*. Springer New York, New York, NY, 413–424. https://doi.org/10.1007/978-1-4614-3185-5_33
- [7] Valerie J Shute. 2008. Focus on formative feedback. *Review of educational research* 78, 1 (2008), 153–189.
- [8] Shin Hwei Tan, Darko Marinov, Lin Tan, and Gary T. Leavens. 2012. @tComment: Testing Javadoc Comments to Detect Comment-Code Inconsistencies. In *2012 IEEE Fifth International Conference on Software Testing, Verification and Validation*. 260–269. <https://doi.org/10.1109/ICST.2012.106>
- [9] Al Vermeulen. 2000. *The elements of Java (tm) style*. Number 15. Cambridge University Press.
- [10] Lin-Jung Wu and Kuo-En Chang. 2023. Effect of embedding a cognitive diagnosis into the adaptive dynamic assessment of spatial geometry learning. *Interactive Learning Environments* 31, 2 (2023), 890–907. <https://doi.org/10.1080/10494820.2020.1815216>